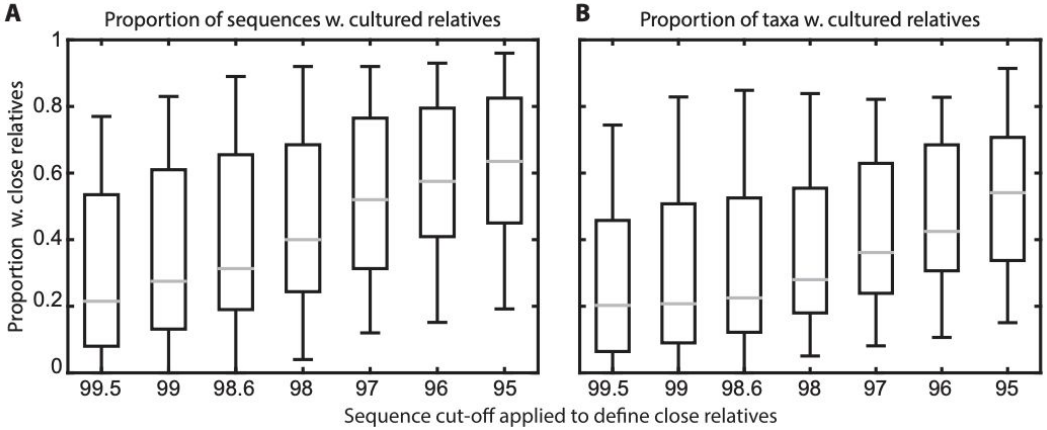
Predicting the Culturability of Marine Bacterial Genomes Using Greedy and Monte Carlo Algorithmic Approaches

Group 3
Juo-Hsuan Chang, Zachary Brown, Juheng Wu
BENG 280B, Winter 2025

Introduction

- 1% paradigm - only 1% of microorganisms (cells or taxa) are culturable
- Recent work suggests otherwise - 35% of taxa have a closely related cultured relative
- Marine bacteria are most useful when cultured



Guiding question: Can we use genetic sequences to predict if a marine bacteria will be 'culturable' or not?

Datasets

- Set A - 1870 16S rRNA genes sequenced from 1561 marine heterotrophic bacterial strains

Research article | Open access | Published: 13 July 2020

Diversity and distribution of marine heterotrophic bacteria from a large culture collection

Isabel Sanz-Sáez, Guillem Salazar, Pablo Sánchez, Elena Lara, Marta Royo-Llonch, Elisabet L. Sà, Teresa Lucena, María J. Pujalte, Dolors Vaqué, Carlos M. Duarte, Josep M. Gasol, Carlos Pedrós-Alió, Olga Sánchez & Silvia G. Acinas

BMC Microbiology **20**, Article number: 207 (2020) | Cite this article

- Set B - 628 16S rRNA genes from uncultured marine bacteria, pulled from NCBI nucleotide search

- Had full genome sets of data available for access if not for memory limitations

Search details

```
(16S[All Fields] AND rRNA[All Fields]) AND "uncultured marine bacterium"[porgn] AND (bacteria[filter] AND biomol_rrna[PROP] AND ddbj_embl_genbank[filter])
```

Problem Formulation

A = { "ACGATCATGGGATCCCCAATG….ACTG",
"GGTCATCAGTCGGATGCAGC….TTCCTA",
"TCTTACTCACACCTGGGTGAT….GGTGAA",
"ACGGCATGGGATCCCCAATG….ATTAGA",
….
"CCCATAAGGATCCCCAATG….AAGGAT"}

B =
{"TTTCTATTCTAACCAA…AATT",
"GGATTGGTTTTGGAAAA…AGGT",
"CCTTATTAACCATTAGG…TCGT",
…
"CGTTGGAGGGCGATTAGG…CGGT",
}

S = {"TCAT", "GGGAT", .}. }

Definitions

- *A*: (culturable genome set)
- *B*: (unculturable genome set)
- *d_match(v, S, d)*: Given a string *v,* a string-set *S*, and a non-negative integer *d*, we say that *v* '*d-matches*' *S* if there exists a substring of *v* that approximately matches one of the strings in *S* with at most *d* mismatches.
- *coverage(S, V, d)*: We define *coverage(S, V, d)* as the fraction of strings *v* in *V* that *d-match S*.
- *score(A, B, S, d)*: Given two sets of strings *A* and *B,* and a set of *k*-mers *S*, define *score(A, B, S, d)* as *coverage(S, A, d) - coverage(S, B, d)*

Input

String-sets *A* and *B*, a positive integer *k*, and a non-negative integer *d*.

Output

A string set *S* maximizing *score(A, B, S, d)* among all sets of *k*-mers.

Goal:

A = { “ACGATCATGGGATCCCCAATG….ACTG”,
“GGTCATCAGTCGGATGCAGC….TTCCTA”,
“TCTTACTCACACCTGGGTGAT….GGTGAA”,
“ACGGCATGGGATCCCCAATG….ATTAGA”,
….
“CCCATAAGGATCCCCAATG….AAGGAT”}

B =
{“TTTCTATTCTAACCAA…AATT”,
“GGATTGGTTTTGGAAAA…AGGT”,
“CCTTATTAACCATTAGG…TCGT”,
…
“CGTTGGAGGGCGATTAGG…CGGT”,
… }

S = {“TCAT”, “GGGAT”, …}

Given the two groups of strings (**A** and **B**), a length for the **k-mers**, and an allowed number of mismatches (**d**),

Find the best **set of k-mers** that clearly separate **A** and **B**—meaning they appear **a lot in A group and very little in the B group**.

Why?

This helps us identify **unique patterns** in **A** vs. **B**, which is useful for tasks like **detecting genetic markers**

This will help us extract genomic fragments that could represent specific features of a cell, in this case, the culturability of a marine bacterium

Count Dictionary Construction

Algorithm 2 Construct Count Dictionary

Input Set of sequences G, range for k-mer length (k_{min}, k_{max}), mismatch threshold d
Output Dictionary *count* mapping k-mers to sequence indices

```
Initialize count ← empty dictionary
for each sequence g_i ∈ G do
    for k = min_k to max_k do
        for each k-mer kmer in g_i do
            for all variant v of kmeer with at most d mismatches do
                Add i to count[v]
            end for
        end for
    end for
end for
return count
```

```
set_G = ["ATCTGT",
         "GTCATTT",
         "CTGGA"]

count_dict = get_count_dict(set_G,
                            min_k=3,
                            max_k=4,
                            d=1)
```

```
Count dictionary:
TTC {0, 1}
AAC {0}
CTC {0, 1, 2}
...
```

Algorithmic Challenge

- Searching for an **optimal set** over **all possible** k-mers candidates:

$$\sum_{i=0}^{|C|} \binom{|C|}{i} = 2^{|C|}$$

Where |C| has a upper bound of $|C| = N \times (L - k + 1)$

Data	k_range	|C|
100 randomly chosen 16S sequences	5-8	75,748
	9-14	1,479,775
	15-18	1,897,387
All 16S sequences	5-8	85,348

N be the number of sequence in set *A*, *L* be the average length of each sequence, In our case, we have around 1500 sequences of real data, and each have length about 600 to 1300 characters. So the brute force algorithm will search over **2 to the power** of a million possible combinations.

Brute Force

Algorithm 1 Brute Force Characteristic K-mer Set

Input Sets of sequences A and B, range for k-mer length (k_{min}, k_{max}), mismatch threshold d
Output Selected k-mer set S

```
C ← ∅                                                    ▷ Initialize an empty set for
for each sequence in A do
    for i ← 0 to length(sequence) − k do
        kmer ← substring(sequence, i, k)
        Add kmer and its variants to C
    end for
end for
bestScore ← −∞
bestSet ← ∅                                              ▷ Best candidate k-
for each candidateSet S in PowerSet(C) do
    currentScore ← score(A, B, S, d)
    if currentScore > bestScore then
        bestScore ← currentScore
        bestSet ← S
    end if
end for
return bestSet
```

Overall runtime: $O(2^{|C|} \times (N+M) \times |C| \times L \times k)$
Due to the exponential number of candidate subsets, this brute-force approach is **computationally infeasible** for large-scale bacterial genome datasets.

Our Proposed Algorithms

- **Greedy Algorithm**
- Monte Carlo
- Modified Greedy Algorithm

Algorithm 3 Greedy Algorithm

Input Count dictionaries *A_cover* and *B_cover*
Output Selected k-mer set S^*

```
Initialize A_cover ← ∅, B_cover ← ∅ and S* ← ∅
while True do
    Initialize gain* ← −inf
    for each k-mer kmer in A_count do
        gain ← (A_count[kmer] − A_cover) − (B_count[kmer] − B_cover)
        if gain is greater than gain* and kmer is not in S* then
            gain* ← gain
            kmer* ← kmer
        end if
    end for
    if gain* > 0 then
        Add kmer* to set S* and Update A_cover & B_cover
    else
        break
    end if
end while
return S*
```

Our Proposed Algorithms

- Greedy Algorithm
- **Monte Carlo**
- Modified Greedy Algorithm

Algorithm 4 Monte Carlo Algorithm

Input Count dictionaries *A_cover* and *B_cover*, negative counter threshold *neg_cnt_thres*, number of trials T
Output Selected k-mer set S^*

```
Initialize S* ← ∅ and best score score* ← −inf
for t = 1 to T do
    Initialize A_cover ← ∅, B_cover ← ∅ and S ← ∅
    Initialize neg_cnt ← 0
    while neg_cnt is less than neg_cnt_thres do
        Randomly sample a k-mer kmer from A_count
        gain ← (A_count[kmer] − A_cover) − (B_count[kmer] − B_cover)
        if gain is greater than 0 then
            Add kmer to set ∗ and Update A_cover & B_cover
            neg_cnt ← 0
        else
            Add 1 to neg_cnt
            if neg_cnt = neg_cnt_thres then
                break
            end if
        end if
    end while
    if score of S is greater than score* then
        S* ← S
        score* ← score of S
    end if
end for
return S*
```

Our Proposed Algorithms

- Greedy Algorithm
- Monte Carlo
- **Modified Greedy Algorithm**

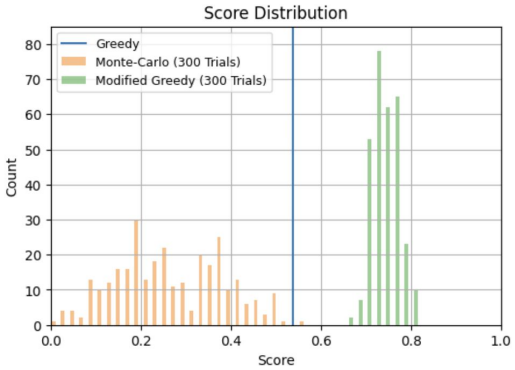
Algorithm 5 Modified Greedy Algorithm

Input Count dictionaries *A_cover* and *B_cover*, threshold θ, number of trials *T*
Output Selected k-mer set S^*

```
Initialize S* ← ∅ and best score score* ← −inf
for t = 1 to T do
    Initialize A_cover ← ∅, B_cover ← ∅ and S ← ∅
    while True do
        gain* ← −inf
        for each k-mer kmer in A_count do
            gain ← (A_count[kmer] − A_cover) − (B_count[kmer] − B_cover)
            if gain is greater than gain* and kmer is not in S* then
                if (a random value from 0-1) > θ then
                    gain* ← gain
                    kmer* ← kmer
                end if
            end if
        end for
        if gain* > 0 then
            Add kmer* to set S* and Update A_cover & B_cover
        else
            break
        end if
    end while
    if score of S is greater than score* then
        S* ← S
        score* ← score of S
    end if
end for
return S*
```

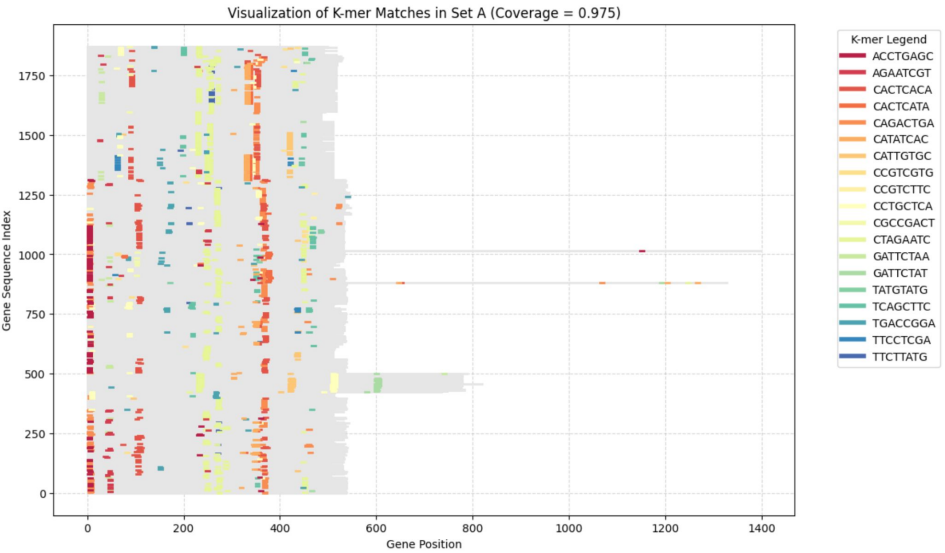
Results: Algorithm Comparisons

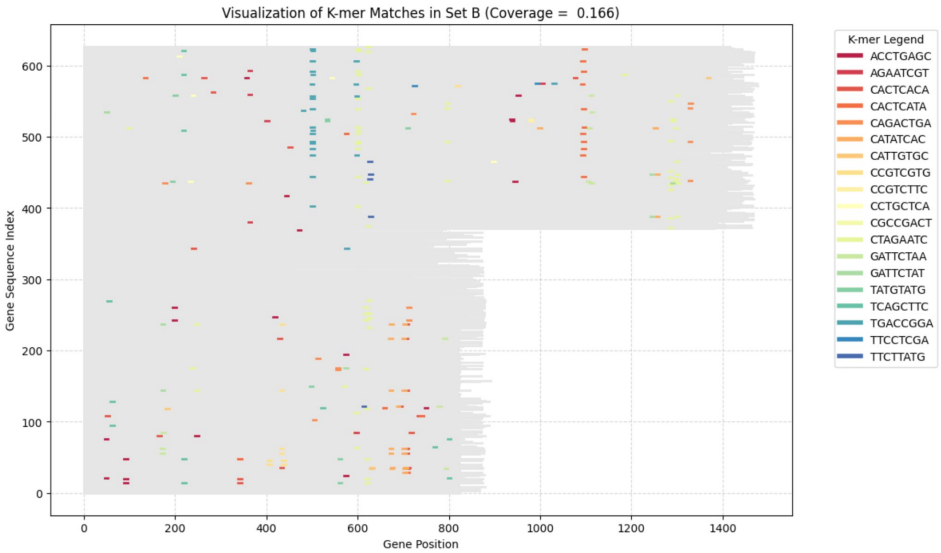
- Best scores achieved with our modified greedy algorithm
- Obtained efficient runtimes with large datasets
- Working with full dataset constrained to kmer range 5-8 by memory



Sets A and B	k_range	d	Algorithm	# of Trials	# of kmers	Runtime	Score
All 16S	5-8	1	Greedy	–	8	0:18	0.538
			Monte Carlo	300	14	3:56	0.570
			Modified Greedy	20	17	2:11	0.810
100 each 16S	5-8	1	Greedy	–	8	0:03	0.80
			Monte Carlo	100	7	1:01	0.65
			Modified Greedy	20	9	0:10	0.90
100 each 16S	9-14	1	Greedy	–	8	0:46	0.97
			Monte Carlo	50	6	29:20	0.89
			Modified Greedy	20	8	2:32	1.0
100 each 16S	15-18	1	Greedy	–	8	0:45	0.830
			Monte Carlo	20	21	19:07	0.940
			Modified Greedy	50	10	09:41	0.990

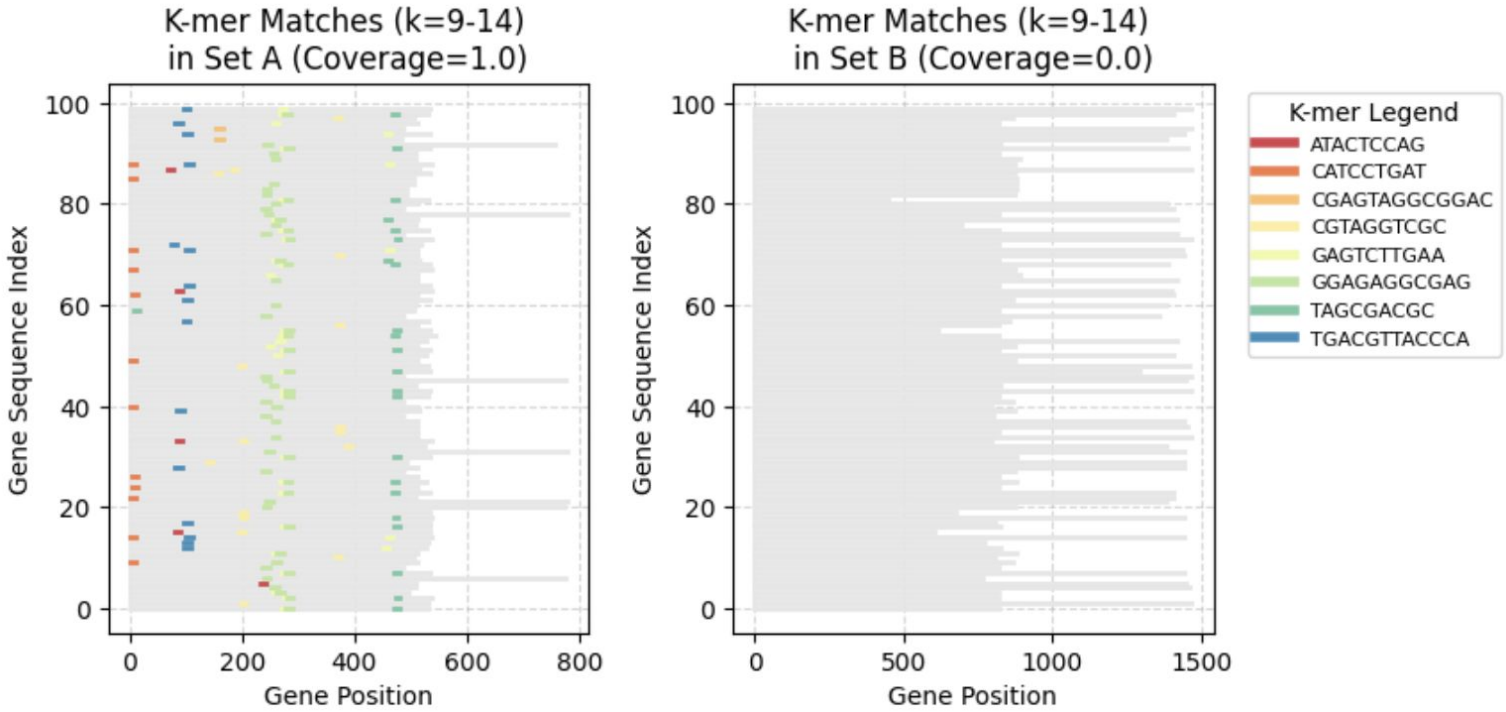
Results: Modified Greedy Algorithm, coverage of full Set A and B, k range 5-8





- Coverage of A = 0.975, B = 0.166
- Gray bar = 16S gene length

Results: Modified Greedy Algorithm, random 100 sequences of Set A and B, k range 9-14



- Coverage of A = 1, B = 0
- Gray bar = 16S gene length

Conclusions

- Developed an efficient algorithm capable of working through large datasets in short timescales (after count dictionary creation)
- Results offer confidence that we can identify biologically relevant sections of genes/genomes that distinguish groups of one phenotype from groups of another
- Memory constraints limited datasets we could work with and k ranges we could explore - solving these issues would allow us to work with more biologically impactful data

References

[1] Gómez, A., Rodrigues, V., Coutinho, E. C. E., Bertrand, R. M. M., and Triplett, E. W. Metagenomic analysis of the honey bee gut microbiome reveals novel bacterial species and metabolic pathways. *BMC Microbiology 20*, 1 (2020), 1–16.

[2] Martinez, J. A., Dethlefsen, L., Lucks, J. G., Goodman, A. L., Ravel, J., Nelson, K. E., Munro, J. B., Turnbaugh, P. J., Gordon, J. I., Huttenhower, C., and Knight, R. The gut microbiome: a new frontier in microbiome research. *Scientific Data 9*, 1 (2022), 1–9.

[3] Martiny, A. C. High proportions of bacteria are culturable across major biomes. *The ISME Journal 13*, 8 (2019), 2125–2128.

[4] Metropolis, N., and Ulam, S. The monte carlo method. *Journal of the American Statistical Association 44*, 247 (1949), 335–341.

[5] Rodrigues, C. J. C., and de Carvalho, C. C. C. R. Cultivating marine bacteria under laboratory conditions: Overcoming the "unculturable" dogma. *Frontiers in Bioengineering and Biotechnology 10* (2022).

[6] Sun, Z.-Z., Ji, B.-W., Zheng, N., Wang, M., Cao, Y., Wan, L., Li, Y.-S., Rong, J.-C., He, H.-L., Chen, X.-L., Zhang, Y.-Z., and Xie, B.-B. Phylogenetic distribution of polysaccharide-degrading enzymes in marine bacteria. *Frontiers in Microbiology 12* (2021).

Our Proposed Algorithms

- **Greedy Algorithm**
- Monte Carlo
- Modified Greedy Algorithm

Algorithm 3 Greedy Algorithm

Input Count dictionaries A_cover and B_cover
Output Selected k-mer set S^*

```
Initialize A_cover ← ∅, B_cover ← ∅ and S* ← ∅
while True do
    Initialize gain* ← −inf
    for each k-mer kmer in A_count do
        gain ← (A_count[kmer] − A_cover) − (B_count[kmer] − B_cover)
        if gain is greater than gain* and kmer is not in S* then
            gain* ← gain
            kmer* ← kmer
        end if
    end for
    if gain* > 0 then
        Add kmer* to set S* and Update A_cover & B_cover
    else
        break
    end if
end while
return S*
```

Our Proposed Algorithms

- Greedy Algorithm
- **Monte Carlo**
- Modified Greedy Algorithm

Algorithm 4 Monte Carlo Algorithm

Input Count dictionaries *A_cover* and *B_cover*, negative counter threshold *neg_cnt_thres*, number of trials T
Output Selected k-mer set S^*

```
Initialize S* ← ∅ and best score score* ← −inf
for t = 1 to T do
    Initialize A_cover ← ∅, B_cover ← ∅ and S ← ∅
    Initialize neg_cnt ← 0
    while neg_cnt is less than neg_cnt_thres do
        Randomly sample a k-mer kmer from A_count
        gain ← (A_count[kmer] − A_cover) − (B_count[kmer] − B_cover)
        if gain is greater than 0 then
            Add kmer to set ∗ and Update A_cover & B_cover
            neg_cnt ← 0
        else
            Add 1 to neg_cnt
            if neg_cnt = neg_cnt_thres then
                break
            end if
        end if
    end while
    if score of S is greater than score* then
        S* ← S
        score* ← score of S
    end if
end for
return S*
```